

Condensed Report: Quantum Peak Tensor-Network Attempts

Codex run on `tree-tensor-sim` worktree

Snapshot compiled on 2026-06-06

Executive Summary

This report condenses the work performed in the current Codex session on the Quantum Peak challenge circuits. The current candidate rollout contains **40 selected answers out of 49 challenge circuits**. The unresolved labels at the report snapshot are:

64_40, 104_49, 48_42, 56_43, 64_44, 72_45, 80_46, 88_47, 96_48.

The strongest completed path so far is a Python/Quimb tensor-network adaptation: graph-weighted qubit ordering followed by `quimb CircuitMPS` simulation and sampling. Exact statevector answers were used where feasible and were always preferred. For several hard circuits, the graph-ordered MPS evidence became too slow or too diffuse, so the session pivoted to the local `peaked-circuit-simulation` MPO iterative cancellation with unswapping. A known-answer validation on 16_28 completed in 93.26 seconds and confirmed the bit-order convention for that method. A clean unresolved MPO-unswapping GPU array, 34614254, is running under a strict two-hour task limit and is not yet reflected in the 40/49 rollout.

What Was Implemented

Worktree and scope. All implementation was done in a new worktree, `quantum-junction-tree-tensor`, on branch `tree-tensor-sim`. Heavy work was run through Slurm. The login node was used only for code editing, file inspection, collection, plotting, and compilation.

Graph-ordered MPS runner. The main completed solver is `jobs/quimb_tree_tensor_runner.py`. It parses QASM with Qiskit, constructs a weighted interaction graph from `cx/swap` operations, derives a qubit order, remaps the circuit, and runs `qiskit_quimb.quimb.circuit(..., CircuitMPS)`. It records:

- the selected backend and package versions,
- interaction graph and ordering metadata,
- MPS bond and tensor-size statistics,
- marginal bitstrings,
- sampled top bitstrings in Qiskit/counts order,
- known-answer validation where available.

The default ordering was weighted spectral. Additional bounded fallback passes used reverse Cuthill-McKee, maximum-spanning-tree DFS, weighted degree, low-bond “fast”, medium-bond, and identity orderings. The fallback results were kept in separate output folders so they could corroborate or fill gaps without overwriting canonical evidence.

Candidate collector. The reportable answers come from `jobs/collect_peak_candidates.py`. It merges exact baselines, GPU MPS, CPU MPS, fallback MPS, Aer pilot evidence, and now MPO-unswapping evidence. It chooses

candidates by source priority first, then validation status and top fraction. The current priority order is exact statevector, GPU graph-MPS, CPU graph-MPS, MPO-unswapping, RCM/MST/degree/mid/fast/identity fallbacks, and finally the older Aer pilot.

MPO-unswapping pivot. After the user noted that attempts running longer than two hours were probably not useful, the long graph-MPS tail was stopped. I then adapted the local `peaked-circuit-simulation` code, which implements the Kremer-Dupuis middle-MPO attack with greedy unswapping. Changes made:

- exposed `sabre.trials` in `unswap.py`, replacing the hard-coded 10000 routing trials with a configurable value;
- added `jobs/peaked_mpo_unswap_runner.py`;
- added `jobs/run_peaked_unswap_gpu_array.slurm`, capped at 02:00:00;
- patched marginal extraction to use current Quimb `compute_local_expectation(..., method="canonical")` after the bundled helper failed on a renamed `partial_trace` API;
- validated bit order on known circuit 16_28; Qiskit/counts order is the final measurement permutation applied and then reversed.

How Answers Were Chosen

All selected answers are in Qiskit/counts order, with the right-most bit corresponding to qubit 0. Exact statevector peaks were selected whenever available. Otherwise, the collector selected the highest-priority tensor-network evidence that produced a candidate and was not known to be incorrect.

For the graph-MPS path, the default candidate is the most frequent sampled bitstring if sampling succeeded; otherwise it falls back to the marginal bitstring. For the MPO-unswapping path, the known-answer validation showed that the candidate should use `permuted_measurement_order_reversed`. This was patched before launching the clean unresolved array.

Known-answer rows were useful for validating bit order and rejecting bad evidence. For example, RCM evidence for 24_29 was low-quality and incorrect, so it was not allowed to replace exact evidence.

Current Results

The rollup selected 40 answers from 49 circuits. The source mix is shown in Figure 1. Exact statevector and GPU MPS dominate the high-confidence rows. CPU MPS fills many very-easy and moderate circuits. RCM and Aer each currently contribute one low-priority fallback row.

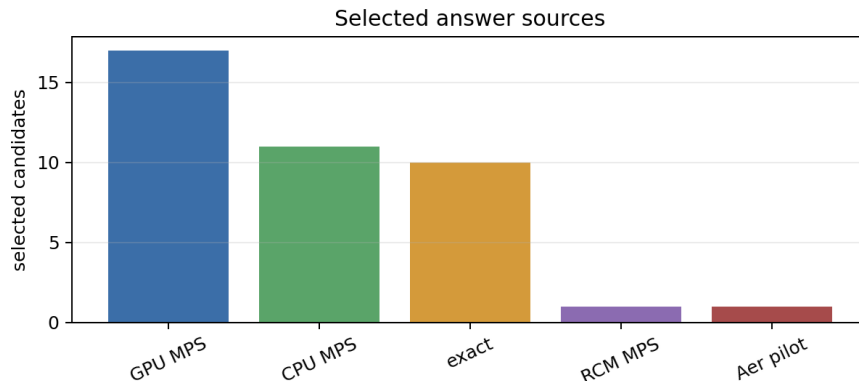


Figure 1: Selected candidate source counts in the current rollup.

Figure 2 summarizes peak strength and runtime for selected evidence. The easy and very-easy circuits usually have a visible sample peak, while several hard rows have very low observed top fractions. Those hard-row candidates should be treated as provisional until corroborated by the MPO-unswapping pass or another stronger method.

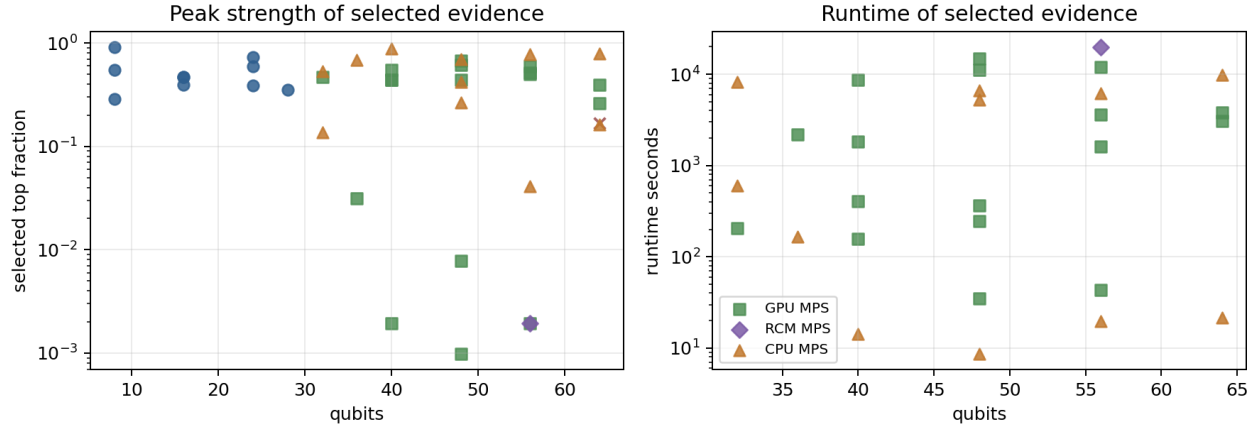


Figure 2: Selected evidence top fraction and runtime. Both axes use logarithmic scaling where appropriate. Exact rows have no runtime in this plot.

Representative Distributions

Figure 3 compares representative sampled distributions. 40_17 is a typical easy case: the peak dominates the sample. 56_39 and 56_38 show why the remaining hard tail was problematic for the MPS approach: the observed top count is only one or two samples out of hundreds or thousands, so the chosen bitstring is much less stable. The 16_28 MPO-unswapping validation shows that the new path can recover the correct known answer quickly after applying the validated bit-order convention.

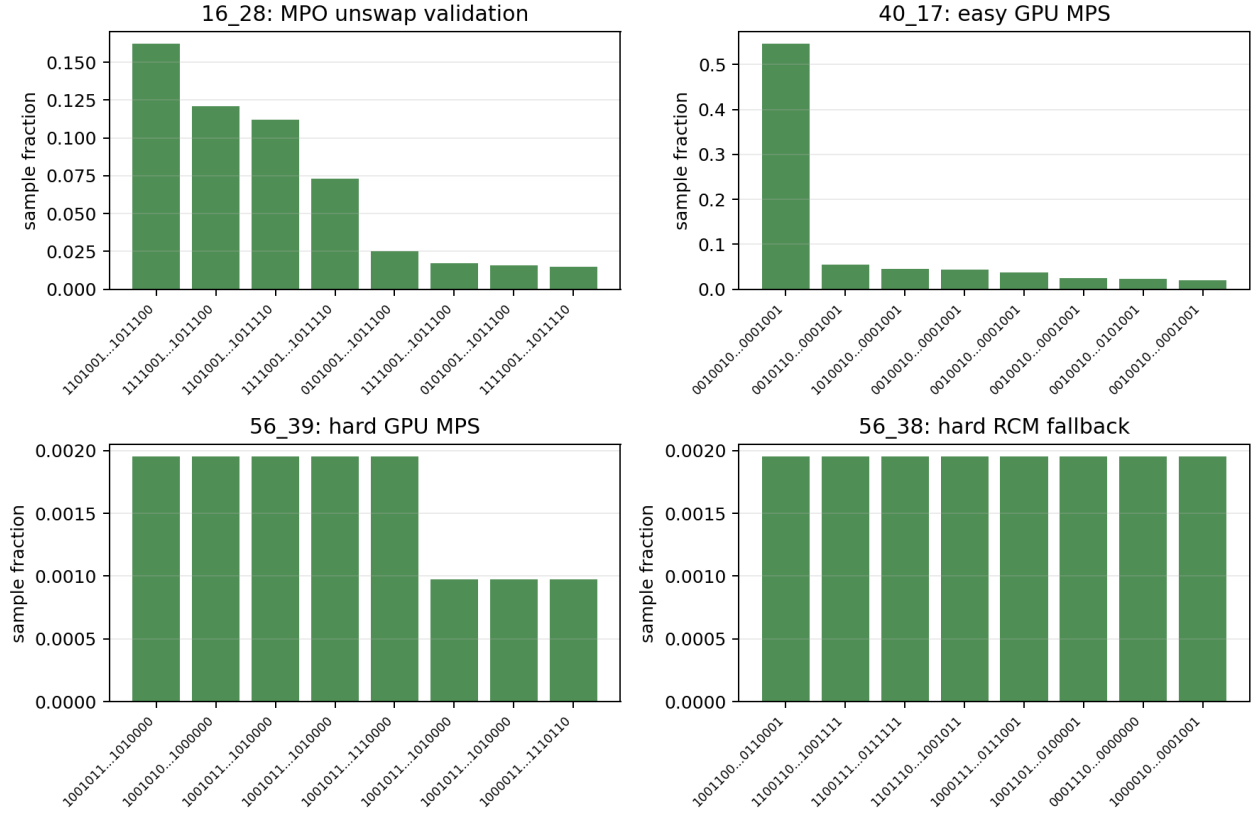


Figure 3: Top sampled bitstring fractions for representative completed runs. Bitstring labels are shortened for readability.

Notable Evidence

- 16_28: exact answer 1101001111011100. The MPO-unswapping validation also produced this answer in 93.26 seconds, confirming the bit-order convention.
- 48_37: GPU and CPU graph-MPS agreed on the same 48-bit answer; the observed top fraction was only 0.0009765625.
- 56_38: first filled by RCM CPU fallback; this is currently a weak fallback with top fraction 0.001953125.
- 56_39: initially filled by degree-order fallback, then replaced by higher-priority GPU graph-MPS; top fraction remains only 0.001953125.
- 64_41: filled only by the older Aer pilot with an unstable classification. It is included because it is currently the only candidate evidence for that label.

Current Caveats

This is not yet a completed challenge solution. The report snapshot is 40/49. The unresolved MPO-unswapping array is active and may change the final rollup. The hard-tail MPS candidates with tiny top fractions are useful as placeholders and diagnostics, but they are not as trustworthy as exact rows or high-top-fraction MPS rows.

The main technical lesson from this session is that naive or graph-ordered MPS is not enough for the very-hard tail. The remaining circuits are dense RX/RZ/CX circuits with no explicit SWAPs, so the useful structure is likely hidden mirrored cancellation/permutation rather than a simple qubit-line ordering. The MPO iterative cancellation with

unswapping is the right next path, and early validation shows it is compatible after the Quimb API and bit-order fixes.

Candidate Tables

The following tables are generated directly from `outputs/tree_tensor_sim/collector_current/CANDIDATES.tsv` at report-build time.

Difficulty	selected	total
easy	16	16
hard	6	7
moderate	8	8
very easy	10	10
very_hard	0	8

Source	selected
GPU MPS	17
CPU MPS	11
exact	10
RCM MPS	1
Aer pilot	1

Table 1: Current candidate rollout. Empty candidate rows are unresolved at report time.

Challenge	difficulty	q	candidate	source	top frac
16.12	easy	16	1111000101101011	exact	0.4665
24.13	easy	24	111110011111001011010001	exact	0.5912
32.14	easy	32	00000101001101000001011111101100	GPU MPS	0.4717
36.15	easy	36	110110011011111111011001011110101111	GPU MPS	0.03125
40.16	easy	40	0101110101001110011000111011100110010110	GPU MPS	0.4424
40.17	easy	40	0010010101010111001001000010001000001001	GPU MPS	0.5459
40.18	easy	40	0100000110010010001101111000111111001110	GPU MPS	0.4395
48.19	easy	48	011001010111101100111110000001011101001110010000	GPU MPS	0.6738
48.20	easy	48	1010101001010010100001101010000001011110010000000	GPU MPS	0.6123
48.21	easy	48	111010101110101011110101000001101000100000001001	GPU MPS	0.4414
56.22	easy	56	111001001001100100111100101101100110000001000101 11111011	GPU MPS	0.5898
56.23	easy	56	010011010011111111000001010011101110111000111111 00001101	GPU MPS	0.4971
56.24	easy	56	10011001001111101111110111010111011010100110010 11110001	GPU MPS	0.5176
64.25	easy	64	001110111111000011011011110101100001000001010011 0111000001111011	GPU MPS	0.3965
64.26	easy	64	011010101010001101011101100001110001011011011001 1100011001100110	GPU MPS	0.2617
8.11	easy	8	01001110	exact	0.5454
40.35	hard	40	0101100110101111111000010101001110001001	GPU MPS	0.001953
48.36	hard	48	001111011011001100110011001000101001101010111000	GPU MPS	0.007812
48.37	hard	48	100101001010001101010100100101001001000101010001	GPU MPS	0.0009766
56.38	hard	56	100110010111100000000100110101011000010010011101 10110001	RCM MPS	0.001953

Challenge	difficulty	q	candidate	source	top frac
56_39	hard	56	100101110011010101011111110101101101101100001010 01010000	GPU MPS	0.001953
64_40	hard	64	—	blank	
64_41	hard	64	000100011101001110011001001111011100000010001101 0001101010110010	Aer pilot	0.167
16_28	moderate	16	1101001111011100	exact	0.3962
24_29	moderate	24	110100010111100001001001	exact	0.3901
32_30	moderate	32	10011000010011110111101111010111	CPU MPS	0.1357
48_31	moderate	48	10110011100010101111111101010111011011000110110	CPU MPS	0.4209
48_32	moderate	48	0111010101110111100011100101010101011101110001110	CPU MPS	0.2656
56_33	moderate	56	11001001100101001111010100100010010101111110111 01010000	CPU MPS	0.04102
64_34	moderate	64	001101010001001100111010111010010100101100101101 1001111011100110	CPU MPS	0.1631
8_27	moderate	8	11001001	exact	0.2879
16_2	very easy	16	1010101011001000	exact	0.467
24_3	very easy	24	011110010000101010001000	exact	0.724
28_4	very easy	28	111111100010101010110110011111	exact	0.3511
32_5	very easy	32	00111000101010100001000000010000	CPU MPS	0.5322
36_6	very easy	36	100110100111101001001101110011001000	CPU MPS	0.6885
40_7	very easy	40	0110111011010001010011111110010011000111	CPU MPS	0.8789
48_8	very easy	48	000100100110111001001111111100101011001110010100	CPU MPS	0.6973
56_9	very easy	56	100101101011001011101001100101100111011001100011 01010100	CPU MPS	0.7812
64_10	very easy	64	001101001011000111001011100110010010110001011111 0110010100101011	CPU MPS	0.7871
8_1	very easy	8	10101101	exact	0.9126
104_49	very hard	104	—	blank	
48_42	very hard	48	—	blank	
56_43	very hard	56	—	blank	
64_44	very hard	64	—	blank	
72_45	very hard	72	—	blank	
80_46	very hard	80	—	blank	
88_47	very hard	88	—	blank	
96_48	very hard	96	—	blank	